

No Silver Bullet: Boosting GaussDB Performance on the 30TB TPC-H Workload

<https://arxiv.org/abs/2608.28352>

1. Problem

- 문제: 30TB급 분산 TPC-H에서 노드 수와 병렬도가 증가할수록 CPU 활용률 저하, shuffle 병목, Bloom Filter 확장성 문제, 비효율적인 query plan이 발생
- 왜 중요한가: 수십 TB 규모에서는 단일 최적화보다 실행 엔진·네트워크·옵티마이저를 함께 개선해야 성능을 확장할 수 있음

2. Method

- Pipeline Execution
 - Volcano 방식의 thread/context-switch overhead를 줄이고 pipeline 기반 실행 + event-driven scheduling 적용
 - Pipeline 내부는 pull, pipeline 간 exchange는 push 방식
 - Q21 단일 노드에서 28% faster
- Scalable Shuffle
 - 기존에는 DOP 증가에 따라 communication channel이 증가해 병목 발생
 - two-level mailbox + node-level quota로 communication overhead 감소
 - DNs=32, DOP=32에서 26.89% 개선
- Unified Bus (UB) + URMA
 - TCP 기반 shuffle을 Huawei UB/URMA로 교체
 - kernel/context-switch를 줄이고 낮은 latency와 높은 bandwidth 활용
 - 7.5TB 실험에서 전체적으로 7.18% 개선
 - 단, shuffle이 적은 일부 query에서는 오히려 성능 저하

- Distributed Bloom Filter
 - Bloom Filter를 단순 실행 단계가 아니라 CBO가 비용까지 고려해 배치
 - distributed/partition-aligned BF streaming으로 MPP 환경에 대응
 - BF-aware CBO 적용 시 전체 TPC-H 실행시간 50~60% 감소
- Near-FK Detection
 - HLL 통계를 이용해 명시되지 않은 PK-FK 관계를 근사적으로 탐지
 - aggregation pushdown 및 불필요한 Bloom Filter 제거에 활용

3. Experiments & Results

- TPC-H 30TB
- 32 servers / 128 DNs / 5,120 CPU cores / 64TB memory
- Composite QphH@30TB: 39,508,107
- Hologres 대비 약 40% 향상
- QphH/vCPU
 - GaussDB: 7,716
 - Hologres: 2,533
 - Exasol: 9,837
- 특히 shuffle, Bloom Filter, near-FK, local exchange를 활용하는 query에서 큰 성능 향상
- 단, 논문 작성 시점 기준 TPC-H 결과는 아직 공식 audit되지 않음

4. Insight

- 핵심 기여: 대규모 DB 성능에는 하나의 silver bullet이 없으며, execution + communication + optimization을 함께 개선해야 함
- 한계점
 - 대부분 기존 기술의 개선 및 조합에 가까움
 - UB는 query 특성에 따라 성능이 악화될 수 있어 adaptive selection이 필요
 - 최종 TPC-H 결과가 unaudited라는 점은 benchmark 비교 시 주의 필요
- 내가 가져갈 포인트

- 분산 DB에서는 개별 컴포넌트의 peak 성능보다 scale-out 과정에서 발생하는 병목 제거가 중요
- CPU scheduling → local exchange → network shuffle → Bloom Filter → optimizer가 서로 연결되어 있으므로 end-to-end 관점에서 최적화해야 함
- 큰 성능 향상은 하나의 혁신보다 여러 작은 최적화의 조합에서 나올 수 있음